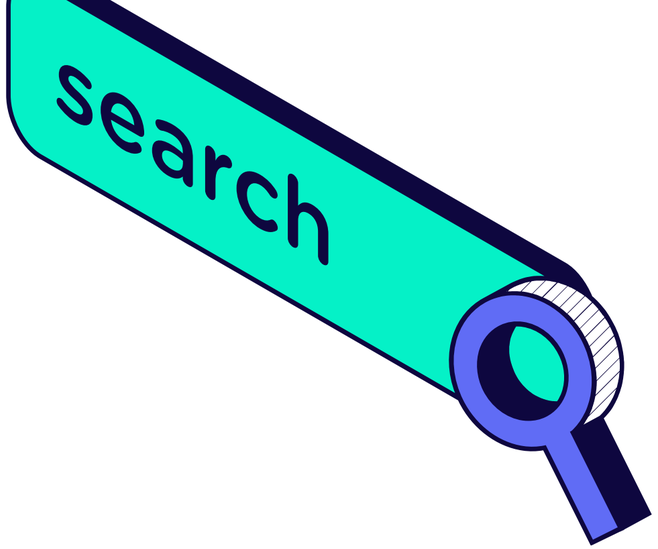
PROJETO DE GERENCIAMENTO DE LIVROS

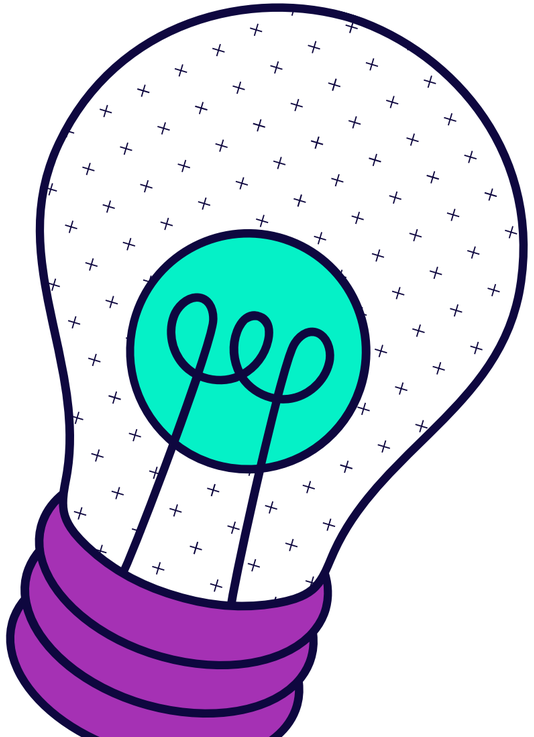
SQLITE3
PYTHON
HTML

ARQUIVOS NA PASTA DO PROJETO

static	PASTA COM OS DOCS DE ESTILO	10/04/2025 21:15
templates	PASTA COM AS JANELAS DO SITE	10/04/2025 21:18
app	ONDE ESTABELECEMOS CONECÇÃO DO “view.py” COM AS JANELAS DO SITE	10/04/2025 22:24
bdados	DOCUMENTO “.db”,OU SEJA, O BANCO DE DADOS CRIANDO EM “bdados.py”	10/04/2025 22:22
bdados		06/04/2025 22:02
view	ONDE CRIAMOS AS FUNÇÕES PARA CONECTAR O BANCO DE DADOS	10/04/2025 22:19



bdados.py



EXPLORER

PROJETO-002 BIBLIOTECA ...

- __pycache__
- .venv
- img
- bdados.db
- bdados.py
- main.py
- view.py

bdados.py

```python
import sqlite3

# Conectar ao banco de dados / criar novo banco de dados
con = sqlite3.connect('bdados.db')

# Criar tabelas

# LIVROS
con.execute('''
    CREATE TABLE IF NOT EXISTS livros(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        titulo TEXT NOT NULL,
        autor TEXT NOT NULL,
        editora TEXT NOT NULL,
        ano DATE)''')
```

Conecta ao banco de dados ‘bdados’, como ele tecnicamente ainda não existe, ele cria automaticamente

Criando a tabela ‘livros’, com os campos:
ID (numero inteiro primária que gera o numero automaticamente)
titulo (texto não pode ser vazio)
autor (texto não pode ser vazio)
editora (texto não pode ser vazio)
ano (formato de data ‘ano-mes-dia’)

```python
# USUÁRIOS
con.execute('''
    CREATE TABLE IF NOT EXISTS usuarios(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nome TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL,
        telefone CHAR(11),
        cpf CHAR(11))''')

# EMPRÉSTIMOS
con.execute('''
    CREATE TABLE IF NOT EXISTS emprestimos(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        livro_id INTEGER,
        usuario_id INTEGER,
        data_emprestimo DATE,
        data_devolucao DATE,
        FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
        FOREIGN KEY (livro_id) REFERENCES livros(id))''')

# Confirmar as alterações no banco de dados
con.commit()
```

OUTLINE

TIMELINE

Criando a tabela ‘usuarios’, com os campos:
ID (numero inteiro chave primária que gera o numero automaticamente)
nome (texto não pode ser vazio)
email (texto não pode ser vazio)
telefone (texto com limite de 11 caracteres)
cpf (texto com limite de 11 caracteres)

Criando a tabela ‘emprestimos’, com os campos:
ID (numero inteiro chave primária que gera o numero automaticamente)
livro_id (numero inteiro)
usuario_id (numero inteiro)
data_emprestimo (formato de data ‘ano-mes-dia’)
data_devolucao (formato de data ‘ano-mes-dia’)
Conecta (usuario_id) com o campo id de ‘usuarios’
Conecta (livro_id) com o campo id de ‘livros’

view.py

EXPLORER

PROJETO-002 BIBLIOTECA ...

- __pycache__
- .venv
- img
- bdados.db
- bdados.py
- main.py
- view.py

view.py

```python
import sqlite3

# conectar o banco de dados
def connect():
    return sqlite3.connect('bdados.db')

# inserir livro
def novo_livro_bd(titulo, autor, editora, ano):
    conn = connect()
    cursor = conn.cursor()  # cursor para manipular os dados

    cursor.execute('''
        INSERT INTO livros (titulo, autor, editora, ano)
        VALUES (?, ?, ?, ?)''', (titulo, autor, editora, ano))
    conn.commit()
    print(f'Livro "{titulo}" cadastrado com sucesso!')

    conn.close()

# Inserir novo usuário
def novo_usuario_bd(nome, email, telefone, cpf):
    conn = connect()
    cursor = conn.cursor()

    # verificar se o usuário já existe
    cursor.execute('SELECT * FROM usuarios WHERE cpf=?', (cpf,))
    if cursor.fetchone():
        print('Já existe um usuário com esse CPF cadastrado.')
    else:
        try:
            cursor.execute('''
                INSERT INTO usuarios (nome, email, telefone, cpf)
                VALUES (?, ?, ?, ?)''', (nome, email, telefone, cpf))
            conn.commit()
            print(f'Usuário "{nome}" cadastrado com sucesso!')
        except sqlite3.Error as e:
            print(f'Já existe um usuário com esse CPF cadastrado.')

    conn.close()
```

OUTLINE

TIMELINE

- Verifica se já existe um usuário com o mesmo CPF.
- fetchone() retorna um resultado (ou None se não houver).
- Se encontrar, imprime mensagem de erro.
- Se não existir, insere o novo usuário.
- O try/except serve como segurança extra caso ocorra algum erro no INSERT.

Serve pra todos de inserir

- conn: conexão com o banco.
- cursor: objeto que permite executar comandos SQL.

- Insere os dados da função nas respectivas tabelas.
- Os “?” são placeholders que previnem SQL Injection. (É como se o invasor enganasse o banco pra executar comandos que não deveriam ser permitidos.)
- Os valores reais são passados como tupla logo depois.

- commit(): grava as mudanças no banco.
- close(): fecha a conexão.

EXPLORER

PROJETO-002 BIBLIOTECA ...

Search (Ctrl+Shift+F)

- .venv
- img
- bdados.db
- bdados.py
- main.py
- view.py

OUTLINE

TIMELINE

view.py

```python
def novo_usuario_bd(nome, email, telefone, cpf):


#exibir livros
def exibir_livros_bd():
    conn = connect()
    cursor = conn.cursor()

    livros = cursor.execute('SELECT * FROM livros').fetchall()
    conn.close()
    return livros


#exibir usuarios
def exibir_usuarios_bd():
    conn = connect()
    cursor = conn.cursor()

    usuarios = cursor.execute("SELECT * FROM usuarios").fetchall()
    conn.close()
    return usuarios



#realizar emprestimos
def realizar_emprestimo_bd(livro_id, usuario_id, data_emprestimo, data_devolucao):
    conn = connect()
    cursor = conn.cursor()

    cursor.execute('''
        INSERT INTO emprestimos (livro_id, usuario_id, data_emprestimo, data_devolucao)
        VALUES (?, ?, ?, ?)''', (livro_id, usuario_id, data_emprestimo, data_devolucao))

    conn.commit()
    print('Empréstimo feito com sucesso!')
    print('\n')
    conn.close()

#exibir emprestimo
def exibir_emprestimos_bd():
    conn = connect()
    cursor = conn.cursor()
```

- Retorna uma lista de todos os livros/usuarios cadastrados.
- fetchall() retorna todos os resultados da consulta como uma lista de tuplas.

- conn: conexão com o banco.
- cursor: objeto que permite executar comandos SQL.

- Insere os dados da função nas respectivas tabelas.
- Os “?” são placeholders que previnem SQL Injection. (É como se o invasor enganasse o banco pra executar comandos que não deveriam ser permitidos.)
- Os valores reais são passados como tupla logo depois.

- commit(): grava as mudanças no banco.
- close(): fecha a conexão.

EXPLORER

PROJET...

- __pycache__
- .venv
- img
- bdados.db
- bdados.py
- main.py
- view.py

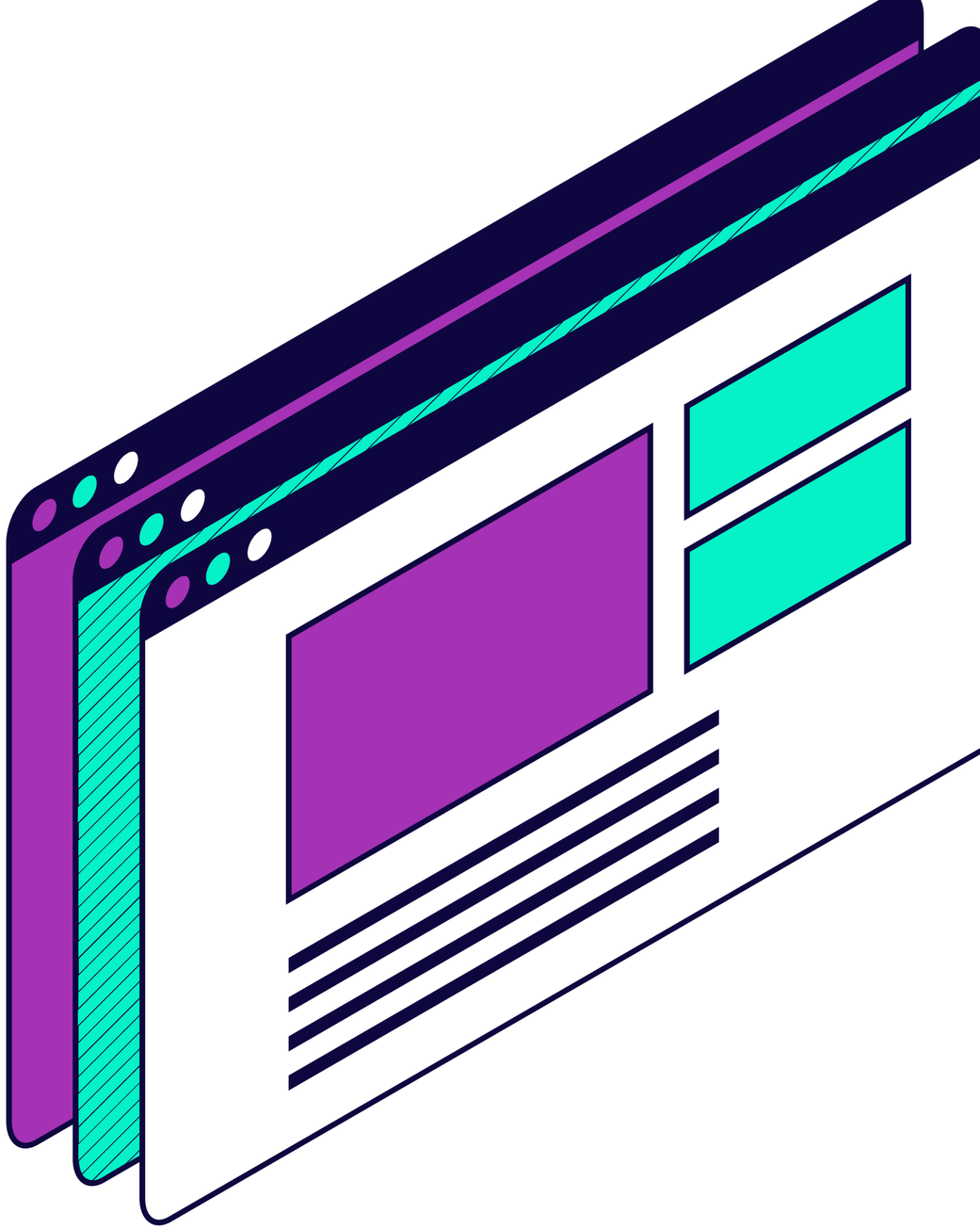
view.py

```python

#exibir emprestimo
def exibir_emprestimos_bd():
    conn = connect()
    cursor = conn.cursor()

    result = cursor.execute('''
        SELECT emprestimos.id, livros.titulo, usuarios.nome, emprestimos.data_emprestimo
        FROM livros
        INNER JOIN emprestimos ON livros.id = emprestimos.livro_id
        INNER JOIN usuarios ON usuarios.id = emprestimos.usuario_id
        WHERE emprestimos.data_devolucao IS NULL
    ''').fetchall()

    conn.close()

    return result

# Atualizar a data de devolução
def atualizar_devolucao_bd(id_emprestimo, data_devolucao):
    conn = connect()
    cursor = conn.cursor()

    cursor.execute('''
        UPDATE emprestimos SET data_devolucao = ? WHERE id = ?''', (data_devolucao, id_emprestimo))

    conn.commit()
    conn.close()

```

- Retorna uma lista de todos os emprestimos feitos cadastrados.
- fetchall() retorna todos os resultados da consulta como uma lista de tuplas.

- Atualiza a data_devolucao do empréstimo com base no id.
- Serve para marcar o livro como devolvido.

app.py



EXPLORER

PROJETO-002 BIBLIOTECA ...

- __pycache__
- .venv
- static
  - css
  - img
- templates
  - cadastrar_livro.html
  - devolucao.html
  - emprestimo.html
  - index.html
  - listar_usuarios.html
  - livros_emprestados....
  - livros.html
  - novo_usuario.html
- app.py
- bdados.db
- bdados.py
- view.py

app.py

```python
from flask import Flask, render_template, request, redirect, url_for
from view import *

app = Flask(__name__)

# Rota principal
@app.route('/')
def index():
    return render_template('index.html')

# Cadastrar novo livro
@app.route('/cadastrar_livro', methods=['GET', 'POST'])
def cadastrar_livro():
    if request.method == 'POST':
        titulo = request.form['titulo']
        autor = request.form['autor']
        editora = request.form['editora']
        data_publicacao = request.form['data_publicacao']

        novo_livro_bd(titulo, autor, editora, data_publicacao)
        return redirect(url_for('exibir_livros'))

    return render_template('cadastrar_livro.html')

# Exibir livros
@app.route('/livros')
def exibir_livros():
    livros = exibir_livros_bd()
    return render_template('livros.html', livros=livros)

```

🔧 **Importações**

- **from flask import ...:** Importa funcionalidades do Flask:

1. Flask: cria a aplicação web.
2. render_template: renderiza arquivos HTML.
3. request: lida com dados enviados pelo formulário.
4. redirect e url_for: redirecionam o usuário para outra rota.

- **from view import *:** Importa todas as funções do módulo view (possivelmente as funções que interagem com o banco de dados).

🚀 **Criação da Aplicação**

- **app = Flask(__name__):** Inicializa o app Flask.

🏠 **Rota Principal**

- **@app.route('/'):** Define a rota da página inicial.
- **render_template('index.html'):** Renderiza o arquivo index.html.

📚 Cadastro de Livros

- **@app.route('/cadastrar_livro', methods=['GET', 'POST']):** Define a rota para cadastrar livros, aceitando requisições GET e POST.
- **if request.method == 'POST':** Se o formulário for enviado: Extrai os dados do formulário (título, autor, editora, data). E Chama a função novo_livro_bd(...) para salvar o livro no banco de dados. Depois Redireciona para a rota /livros após o cadastro.
- **return render_template('cadastrar_livro.html'):** Se for GET, mostra o formulário.

📖 Exibição de Livros

- **@app.route('/livros'):** Rota para exibir os livros cadastrados.
- **livros = exibir_livros_bd():** Busca os livros do banco de dados.
- **render_template('livros.html', livros=livros):** Renderiza a página com os livros.

EXPLORER

app.py

PROJETO-002 BIBLIOTECA ...

- __pycache__
- .venv
- static
  - css
  - img
- templates
  - cadastrar_livro.html
  - devolucao.html
  - emprestimo.html
  - index.html
  - listar_usuarios.html
  - livros_emprestados....
  - livros.html
  - novo_usuario.html
- app.py
- bdados.db
- bdados.py
- view.py

app.py

```python

# Cadastrar novo usuário
@app.route('/novo_usuario', methods=['GET', 'POST'])
def novo_usuario():
    if request.method == 'POST':
        nome = request.form['nome']
        cpf = request.form['cpf']
        telefone = request.form['telefone']
        email = request.form['email']

        novo_usuario_bd(nome, email, telefone, cpf)

    return render_template('novo_usuario.html')

# Listar usuários
@app.route('/listar_usuarios')
def listar_usuarios():
    usuarios = exibir_usuarios_bd()
    return render_template('listar_usuarios.html', usuarios=usuarios)

# Empréstimo
@app.route('/emprestimo', methods=['GET', 'POST'])
def realizar_emprestimo():
    if request.method == 'POST':
        usuario_id = request.form['usuario']
        livro_id = request.form['livro']
        data_emprestimo = request.form['data_emprestimo']
        data_devolucao = None  # Considerando que ainda não foi devolvido

        realizar_emprestimo_bd(livro_id, usuario_id, data_emprestimo, data_devolucao)

    return render_template('emprestimo.html')
```

👤 Cadastro de Novo Usuário

- **@app.route('/novo_usuario', methods=['GET', 'POST']):** Define rota para cadastrar um novo usuário.
- **if request.method == 'POST':** Se o formulário for enviado:
  - Coleta os dados do formulário (nome, cpf, telefone, email).
  - Chama a função novo_usuario_bd(...) para salvar o usuário no banco de dados.
- **render_template('novo_usuario.html'):** Se for GET (ou após salvar), mostra o formulário de cadastro.

📋 Listagem de Usuários

- **@app.route('/listar_usuarios'):** Rota para exibir todos os usuários cadastrados.
- **usuarios = exibir_usuarios_bd():** Recupera os dados dos usuários do banco de dados.
- **render_template('listar_usuarios.html', usuarios=usuarios):** Exibe a lista na página HTML.

Empréstimo de Livro

- **@app.route('/emprestimo', methods=['GET', 'POST']):** Rota para registrar um empréstimo.
- **if request.method == 'POST':** Se o formulário for enviado:
  - Coleta o usuario_id, livro_id, e a data_emprestimo.
  - Define data_devolucao como None (ainda não devolvido).
  - Chama a função realizar_emprestimo_bd(...) para registrar o empréstimo no banco.
- **render_template('emprestimo.html'):** Se for GET (ou após o POST), mostra o formulário de empréstimo.

EXPLORER

app.py

PROJETO-002 BIBLIOTECA ...

- __pycache__
- .venv
- static
  - css
  - img
- templates
  - cadastrar_livro.html
  - devolucao.html
  - emprestimo.html
  - index.html
  - listar_usuarios.html
  - livros_emprestados....
  - livros.html
  - novo_usuario.html
- app.py
- bdados.db
- bdados.py
- view.py

app.py

```python

# Devolução
@app.route('/devolucao', methods=['GET', 'POST'])
def devolucao():
    if request.method == 'POST':
        id_emprestimo = request.form['emprestimo']
        data_devolucao = request.form['data_devolucao']

        atualizar_devolucao_bd(id_emprestimo, data_devolucao)

    return render_template('devolucao.html')

# Livros emprestados no momento
@app.route('/livros_emprestados')
def livros_emprestados():
    emprestimos = exibir_emprestimos_bd()
    return render_template('livros_emprestados.html', emprestimos=emprestimos)

if __name__ == '__main__':
    app.run(debug=True)

```

🔁 Devolução de Livro

- **@app.route('/devolucao', methods=['GET', 'POST']):** Rota para registrar a devolução de um livro.
- **if request.method == 'POST':** Quando o formulário é enviado:
  - Coleta o id_emprestimo (identificador do empréstimo) e a data_devolucao.
  - Chama atualizar_devolucao_bd(...) para atualizar a devolução no banco.
- **render_template('devolucao.html'):** Se for GET (ou após registrar), mostra o formulário de devolução.

📚 Listar Livros Emprestados no Momento

- **@app.route('/livros_emprestados'):** Rota para exibir os empréstimos ativos.
- **emprestimos = exibir_emprestimos_bd():** Busca no banco os livros atualmente emprestados (sem data de devolução).
- **render_template('livros_emprestados.html', emprestimos=emprestimos):** Mostra esses empréstimos na tela.

🚀 Inicialização do Servidor

- **if __name__ == '__main__':**: Garante que o app só será executado se o arquivo for executado diretamente (e não importado).
- **app.run(debug=True):** Inicia o servidor Flask com debug=True (mostra erros no navegador e recarrega automaticamente ao alterar o código).